Поиск пути в лабиринте

Цель работы: познакомиться с применением графов и их обходов к поиску пути; реализовать поиск в глубину и поиск в ширину на неявно заданном графе клеточного лабиринта, сопоставить найденные пути и число просмотренных клеток и экспериментально подтвердить линейную сложность обходов.

Теоретические сведения

Графы

Граф задаётся множеством вершин V и множеством рёбер E, соединяющих пары вершин. Ребро, соединяющее x и y, в неориентированном графе записывается как множество {x, y}: отношение симметрично. В ориентированном графе ребро (x, y) не гарантирует наличия ребра (y, x). Пример ориентированного графа — веб: из страницы u идёт ребро в страницу v, если на u есть ссылка на v. В таких графах миллиарды вершин и рёбер, поэтому алгоритмы на них должны работать за время, близкое к линейному от их размера.

Представление графа. Матрица смежности n × n хранит в клетке (i, j) признак ребра между вершинами i и j: наличие ребра проверяется за O(1), но память — $O(n^2)$, что для больших разреженных графов неприемлемо. Список смежности хранит для каждой вершины перечень её соседей: память O(|V| + |E|), перебор соседей вершины — за число соседей, зато проверка конкретного ребра требует просмотра списка. Обходы перебирают соседей, поэтому для них естественен список смежности.

Лабиринт как неявный граф

Граф не обязательно хранить. Клеточный лабиринт — это граф, где вершины — свободные клетки, а две клетки соединены ребром, если у них общая сторона. Вершина обозначается не номером, а парой координат (строка, столбец), и списки смежности не строятся: соседей клетки (r, c) получают по координатам — (r − 1, c), (r + 1, c), (r, c − 1), (r, c + 1), если клетка внутри поля и не стена. Обходу этого достаточно: ему нужно только уметь по вершине перечислить соседей.

```
for d in ORDER:                 # например, ORDER = "RDLU"
    nr, nc = r + DR[d], c + DC[d]
    if inside(nr, nc) and grid[nr][nc] != WALL:
        ...                     # сосед (nr, nc)
```

У клеточного лабиринта каждая вершина имеет не больше четырёх соседей, поэтому |E| ≤ 2|V| и сложность O(|V| + |E|) здесь — это O(|V|), линейная от числа клеток.

Поиск в глубину

Поиск в глубину (DFS) отвечает на вопрос, какие вершины достижимы из заданной. Он похож на обход лабиринта с клубком ниток и мелом: мелом помечаем, где уже были, чтобы не ходить кругами, а нить позволяет вернуться назад из тупика. Нить — это стек: идя в новый коридор, мы кладём его в стек, возвращаясь — снимаем. В рекурсивной записи роль стека играет стек вызовов:

```
dfs(v):
    отметить v
    если v — выход: успех
    для каждого соседа u клетки v (в порядке ORDER):
        если u не отмечена и dfs(u): успех
    неудача
```

Когда поиск доходит до выхода, содержимое стека — ходы от старта до текущей клетки — и есть найденный путь. Но этот путь не обязан быть кратчайшим: DFS уходит по первому доступному коридору до упора и находит выход тем путём, каким случайно туда пришёл. Какой путь получится, зависит от **порядка перебора соседей**: при разных порядках DFS на одном лабиринте находит разные пути разной длины.

Время работы — $O(|V| + |E|)$: каждая вершина отмечается один раз, каждое ребро просматривается с двух концов. Память — $O(|V|)$ на отметки и на стек, глубина которого в худшем случае равна числу клеток: например, в лабиринте-змейке, где все клетки выстроены в один коридор.

Поиск в ширину и волновой алгоритм

Поиск в ширину (BFS) обходит вершины в порядке удаления от старта: сначала сам старт, затем все клетки на расстоянии 1, затем на расстоянии 2 и так далее. Для клеточного поля его часто называют **волновым алгоритмом**: от старта распространяется волна, на каждом шаге пополняясь свободными клетками, соседними с фронтом и ещё не попавшими в волну. Каждая клетка помечается номером шага, на котором её достигла волна, — это и есть расстояние от старта.

Вместо поиска клеток с меткой d на каждом шаге используется **очередь**: клетки извлекаются из её начала, а новые соседи добавляются в конец, поэтому клетки на расстоянии d всегда обрабатываются раньше клеток на расстоянии $d + 1$. Клетку отмечают в момент постановки в очередь, иначе одна клетка может попасть в очередь несколько раз. Для каждой клетки запоминают, из какой клетки и каким ходом в неё пришли.

Когда волна дошла до выхода, путь восстанавливается **обратным ходом**: от выхода по запомненным ходам назад до старта. Полученный путь — кратчайший: если бы существовал более короткий, волна дошла бы до выхода раньше. Время и память BFS — тоже $O(|V| + |E|)$, но память занимает не стек, а очередь, длина которой определяется шириной фронта волны.

	Поиск в глубину	**Поиск в ширину**
Структура	стек (или рекурсия)	очередь
Найденный путь	какой-то, зависит от порядка соседей	кратчайший по числу ходов
Время	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$
Худший случай по памяти	глубокий стек: длинный коридор-змейка	широкая очередь: открытое поле

Рекурсия и явный стек

DFS можно записать рекурсивно, а можно с **явным стеком**, хранящим для каждой клетки на текущем пути, какого соседа она проверит следующим. Такой стек в точности повторяет то, что при рекурсии хранит стек вызовов, и порядок обхода остаётся тем же. Распространённая ошибка — класть в стек сразу всех непомеченных соседей и помечать их при укладке: такой обход тоже идёт «в глубину», но порядок посещения клеток и найденный путь у него другие.

Взвешенные клетки: Дейкстра и A*

Если проход через клетку имеет стоимость (дорога, болото, лес), кратчайший по числу ходов путь перестаёт быть самым дешёвым. Алгоритм Дейкстры обобщает BFS: вместо очереди используется **очередь с приоритетом** (куча), из которой всегда извлекается клетка с наименьшей известной стоимостью пути от старта; при равных стоимостях всех клеток он совпадает с BFS. A* добавляет к стоимости пути оценку оставшегося расстояния до выхода — эвристику. На клеточном поле без диагоналей естественная эвристика — манхэттенское расстояние $|r_1 - r_2| + |c_1$

− c_2|; она не превышает настоящей стоимости остатка пути (при стоимости клеток не меньше 1), то есть допустима, и A* находит оптимальный путь, просматривая меньше клеток.

Условие задачи

Дан лабиринт на клетчатом поле с одним стартом и одним выходом. Требуется найти путь от старта до выхода поиском в глубину и поиском в ширину, вывести для обоих длину пути, число просмотренных клеток и сам путь, построить графическое представление лабиринта с найденными путями и сопоставить результаты обходов.

Формат входных данных

Лабиринт задаётся числами через пробел — формат прежних лет: 4 — стена, 2 — проход, 10 — старт, 9 — выход. Первой строкой идёт порядок перебора соседей — перестановка букв U (вверх), D (вниз), L (влево), R (вправо). Пример:

```
ORDER RDLU
4 4 4 4 4 4 4
4 10 2 2 2 2 4
4 2 4 4 4 2 4
4 2 2 2 4 2 4
4 4 4 2 2 9 4
4 4 4 4 4 4 4
```

Ответ программы — две строки: для каждого обхода длина пути в ходах, число просмотренных клеток и путь буквами ходов:

```
DFS 7 8 RRRRDDD
BFS 7 14 RRRRDDD
```

Точные правила — в файле `io-format.md` в каталоге работы: в какой момент клетка считается просмотренной, когда обход останавливается, как вывести недостижимый выход, какие ошибки входа распознаются. Он является частью задания: при фиксированном порядке соседей результат DFS однозначен, и программа, расходящаяся с правилами в мелочах, не проходит автоматическую проверку.

Базовые требования к программе

- Обходы реализованы самостоятельно; библиотеки алгоритмов на графах не используются. Очередь и стек — любые, включая стандартные.
- Программа выполняет оба обхода на одном лабиринте и выводит результаты по `io-format.md`.
- Программа строит графическое представление лабиринта с путями DFS и BFS и сохраняет его в файл; режим рисования включается отдельным флагом и не меняет проверяемый выходной файл. Формат изображения — любой; BMP можно записать самостоятельно, как в работе 2.
- Программа не завершается аварийно и не зависает на некорректном входе: пустом, двоичном, с неверным заголовком, рваными строками, лишними или отсутствующими стартом и выходом.
- Предусмотрен режим замера, выводящий время каждого обхода отдельно от чтения файла.

Формат запуска фиксирован, поскольку проверка автоматическая:

```
solution лабиринт.txt ответ.txt
```

Проверка результата

Перед сдачей прогоните автоматическую проверку из каталога репозитория:

```
python3 check.py r4-maze
```

Публичные тесты включают лабиринт из прежней версии работы, один лабиринт с четырьмя разными порядками обхода, пустую комнату, недостижимый выход, поле без внешней стены и ошибки формата. Тесты устойчивости проверяют, что программа не падает на враждебном входе. Вместе с вариантом выдаётся персональный лабиринт `personal.in` со своим порядком обхода; вывод на нём сдаётся вместе с отчётом и сверяется с эталоном. При приёмке добавляются скрытые тесты.

Замеры

В каталоге `bench/` лежат лабиринты, площадь которых удваивается от 8 × 8 до 512 × 512 клеток лабиринта. Измерьте время каждого обхода отдельно от чтения файла по протоколу курса и постройте график от числа клеток поля. Сопоставьте форму графика с оценкой O(|V| + |E|). Если на каком-то размере программа перестала работать, это результат: объясните в отчёте, что именно отказало и почему.

Дополнительное задание (повышенный уровень)

Выбрать не менее двух пунктов.

- **Явный стек.** Переписать поиск в глубину на явный стек — стек из работы 1 — так, чтобы результат в точности совпадал с рекурсивным, и пройти большие лабиринты из `tests/large-public`. Оценить предельный размер лабиринта для рекурсивной и итеративной версий.
- **Стоимость клетки и алгоритм Дейкстры.** Назначить клеткам стоимость прохода, найти путь минимальной стоимости алгоритмом Дейкстры на куче из работы 3 и сравнить с путём BFS. Формат задания стоимостей описать в отчёте.
- **A*.** Добавить манхэттенскую эвристику и сравнить число просмотренных клеток у BFS, Дейкстры и A* на одних и тех же лабиринтах. Обосновать допустимость эвристики.
- **Анимация.** Анимация процесса поиска (фронт волны BFS, продвижение и откаты DFS) и движения по найденному пути — последовательность кадров или видеофайл.

Требования к отчёту

Отчёт оформляется по общему шаблону курса и содержит четыре обязательных раздела.

- **Асимптотика** обоих обходов по времени и памяти через |V| и |E| с обоснованием; почему для клеточного лабиринта это O(|V|).
- **Замеры** по протоколу курса на лабиринтах из `bench/`: не менее шести размеров, медиана, разброс, график и проверка гипотезы роста нормировкой t(n)/f(n).
- **Худший случай** по памяти: какой лабиринт даёт самый глубокий стек DFS и самую длинную очередь BFS; конструкция входа и замер.
- **Масштабирование:** что откажет первым при росте лабиринта на три порядка.

Отдельным пунктом — сопоставление DFS и BFS на персональном лабиринте: длины путей, число просмотренных клеток, изображение обоих путей и объяснение, почему они различаются.

Контрольные вопросы

- Почему BFS находит кратчайший путь, а DFS — нет? Приведите лабиринт, где путь DFS длиннее кратчайшего в несколько раз.
- Почему в BFS клетку отмечают при постановке в очередь, а не при извлечении? Что изменится, если отмечать при извлечении?
- Как зависит путь DFS от порядка перебора соседей? Может ли порядок изменить длину пути BFS? А сам путь BFS?
- Как переписать рекурсивный DFS на явный стек, сохранив порядок обхода? Почему «положить в стек всех соседей сразу» даёт другой обход?
- Какой лабиринт даёт максимальную глубину рекурсии DFS при заданном числе клеток? Какой — максимальную длину очереди BFS?
- В лабиринте без циклов путь между двумя клетками единственный. Чем тогда отличаются результаты DFS и BFS?
- Сколько рёбер в графе клеточного лабиринта с V свободными клетками? Почему матрица смежности для него не годится?
- Чем алгоритм Дейкстры отличается от BFS и когда они дают одинаковый результат? Зачем ему куча?
- Когда эвристика A* допустима и что будет, если взять недопустимую?